

LÝ THUYẾT ĐỒ THỊ

CHIẾN THUẬT QUÉT SẠCH T-MATH

Giảng Viên: Đặng Xuân Lâm

Hành trình chinh phục

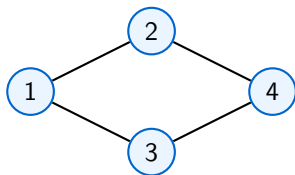
- 1 Tổng quan Đồ thị
- 2 Lưu trữ (Memory Optimization)
- 3 Thuật toán Tìm kiếm
- 4 Bài tập Cách giải (T-Math Style)
- 5 Thuật toán Nâng cao
- 6 ĐẤU TRƯỜNG TRÍ TUỆ (GAME)

Đồ thị là gì?

Định nghĩa cốt lõi

Đồ thị $G = (V, E)$ gồm tập các đỉnh (Vertices) và tập các cạnh (Edges) kết nối các cặp đỉnh.

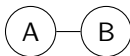
- Đỉnh: Tài khoản Facebook.
- Cạnh: Kết bạn, nhắn tin.



Phân loại: Có hướng & Vô hướng

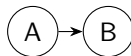
Vô hướng

Đường 2 chiều. Bạn bè trên FB là vô hướng.



Có hướng

Đường 1 chiều. Follow trên TikTok là có hướng.



Lưu trữ: Danh sách kề (Adjacency List)

Vũ khí tối ưu

Tiết kiệm bộ nhớ khi đồ thị có 10^5 đỉnh. Duyệt cực nhanh!

```
vector<int> adj[100005];  
void add_edge(int u, int v) {  
    adj[u].push_back(v);  
    adj[v].push_back(u);  
}
```

Lưu trữ: Ma trận kề (Adjacency Matrix)

- Dùng mảng 2 chiều $A[N][N]$.
- **Ưu điểm:** Kiểm tra u, v có kề nhau không trong $O(1)$.
- **Nhược điểm:** Tốn bộ nhớ $O(N^2)$, dễ bị **MLE** nếu $N > 10^4$.

DFS: Đi xuyên bóng tối (Depth First Search)

Chiến thuật Đệ quy

Ưu tiên đi sâu nhất có thể. Dùng **Stack** hoặc **Đệ quy**.

- Ứng dụng: Tìm thành phần liên thông, kiểm tra chu trình.
- Độ phức tạp: $O(V + E)$.

BFS: Lan tỏa sức mạnh (Breadth First Search)

Chiến thuật Hàng đợi

Quét theo từng lớp (loang). Dùng **Queue**.

- Ứng dụng: Tìm **đường đi ngắn nhất** trên đồ thị không trọng số.
- Độ phức tạp: $O(V + E)$.

Bài toán 1: Đếm số vùng liên thông

- **Bài toán:** Đếm số "đảo" trong đồ thị.
- **Cách giải:** 1. Duyệt tất cả các đỉnh. 2. Nếu đỉnh i chưa thăm, tăng biến đếm và gọi DFS/BFS(i) để đánh dấu toàn bộ đảo đó.

Bài toán 2: Mê cung 2D

Mẹo xử lý

Dùng mảng hướng để code cực nhanh: $dx[] = \{-1, 1, 0, 0\}$, $dy[] = \{0, 0, -1, 1\}$.

S			
			E

Dijkstra: Đường đi ngắn nhất

Tham lam (Greedy)

Luôn chọn đỉnh có khoảng cách nhỏ nhất để tối ưu tiếp.

Độ phức tạp: $O(E \log V)$ khi dùng Priority Queue.

Kruskal: Cây khung nhỏ nhất (MST)

- Sắp xếp các cạnh theo trọng số tăng dần.
- Dùng **DSU (Disjoint Set Union)** để kiểm tra chu trình.
- Gộp các đỉnh lại cho đến khi đủ $V - 1$ cạnh.

Mẹo đi thi: Tăng tốc code

Để tránh bị **TLE** vô lý trên hệ thống T-Math:

```
ios_base::sync_with_stdio(0);  
cin.tie(0); cout.tie(0);  
// Dung '\n' thay vì endl
```

GAME: PHÁ ĐẢO (Câu 1/5)

Thử thách

Để tìm đường ngắn nhất trên đồ thị **không trọng số**, ta dùng thuật toán nào?

GAME: PHÁ ĐẢO (Câu 1/5)

Thử thách

Để tìm đường ngắn nhất trên đồ thị **không trọng số**, ta dùng thuật toán nào?

Đáp án: BFS (Hàng đợi).

GAME: PHÁ ĐẢO (Câu 2/5)

Thử thách

Đồ thị 10^5 đỉnh, dùng ma trận kề sẽ bị lỗi gì?

GAME: PHÁ ĐẢO (Câu 2/5)

Thử thách

Đồ thị 10^5 đỉnh, dùng ma trận kề sẽ bị lỗi gì?

Đáp án: MLE (Memory Limit Exceeded - Tràn bộ nhớ).

GAME: PHÁ ĐẢO (Câu 3/5)

Thử thách

Thuật toán DFS thường được cài đặt bằng cách sử dụng?

GAME: PHÁ ĐẢO (Câu 3/5)

Thử thách

Thuật toán DFS thường được cài đặt bằng cách sử dụng?

Đáp án: Ngăn xếp (Stack) hoặc Đệ quy.

GAME: PHÁ ĐẢO (Câu 4/5)

Thử thách

Kruskal dùng để tìm cái gì?

GAME: PHÁ ĐẢO (Câu 4/5)

Thử thách

Kruskal dùng để tìm cái gì?

Đáp án: Cây khung nhỏ nhất (Minimum Spanning Tree).

GAME: PHÁ ĐẢO (Câu 5 - BOSS)

Thử thách

Độ phức tạp tối ưu của Dijkstra là bao nhiêu?

GAME: PHÁ ĐẢO (Câu 5 - BOSS)

Thử thách

Độ phức tạp tối ưu của Dijkstra là bao nhiêu?

Đáp án: $O(E \log V)$.

- Luôn vẽ đồ thị ra giấy trước khi gõ code.
- Thành thạo DFS/BFS là nắm chắc 50
- Luyện tập tại T-Math để "out trình" đối thủ!

CHÚC MAY MẮN!

Hẹn gặp lại các bạn ở đỉnh vinh quang T-Math!